

1. Architettura del Sistema e il BUS

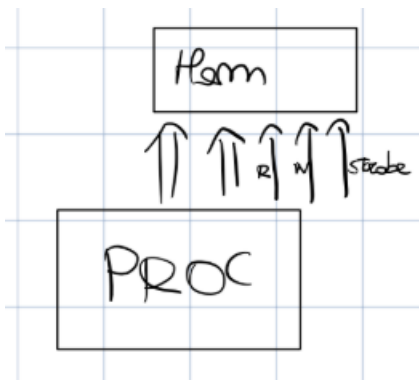
Il sistema è un'architettura digitale che deve interfacciarsi con un mondo analogico. Il collegamento tra i componenti avviene tramite il **BUS**, che è sempre di tipo **parallelo**.

Categorie di segnali sul BUS

Ogni BUS è composto da diversi "fili" dedicati a funzioni specifiche:

- **Alimentazione:** Tensione e massa.
- **Dati:** Possono essere a 8, 16 o 32 bit.
- **Indirizzi:** Per identificare la destinazione delle informazioni.
- **Stato:** Informazioni di ritorno dalle periferiche (es. se sono pronte).
- **Controllo:** Segnali inviati dal processore.

Il processore dialoga con le memorie e le periferiche principalmente attraverso tre segnali logici che viaggiano sul BUS: **Read**, **Write** e **Strobe**.



2. Gestione delle Periferiche I/O

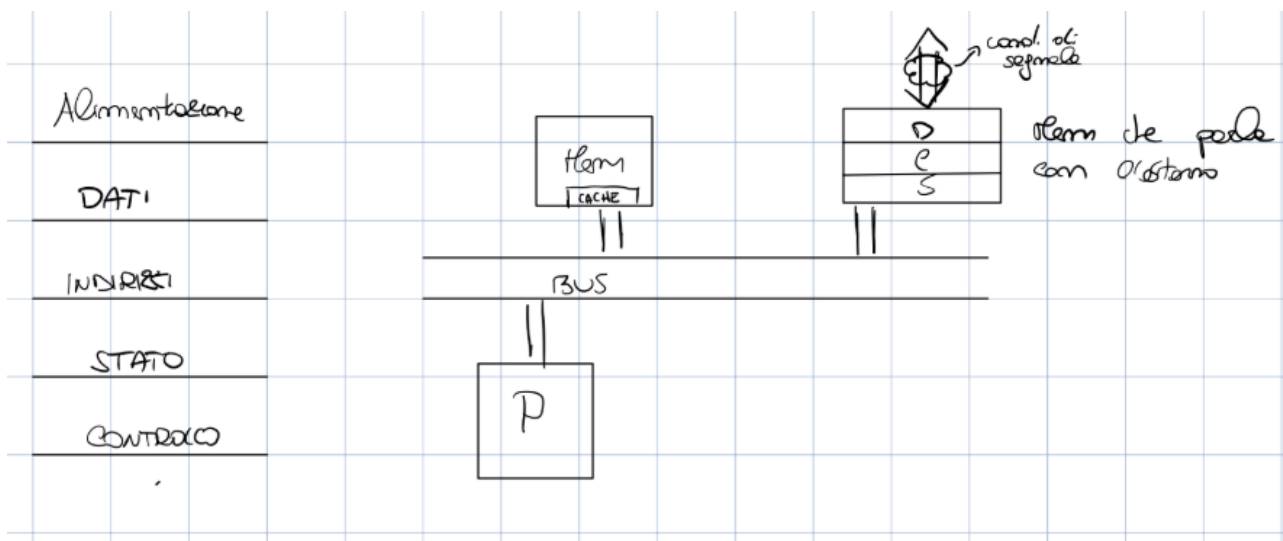
Le periferiche sono oggetti esterni che si connettono all'architettura classica (CPU e memoria). Per funzionare, necessitano di un **condizionamento di segnale** che adatti i segnali del mondo esterno per essere letti o scritti.

L'Interfaccia (Registri)

La memoria non consente di effettuare scambio di informazioni. È un sistema chiuso atto a contenere soltanto i dati. Serve un'altra memoria che dialoga con l'esterno: ogni periferica interagisce con la CPU tramite un'interfaccia composta da **3 registri**:

1. **Data:** Per lo scambio effettivo dei dati.
2. **Controllo:** Usato dalla CPU per gestire il dispositivo.
3. **Stato:** Usato dalla CPU per monitorare le informazioni del dispositivo.

Un esempio di periferica citato è la **UART**, che trasmette dati in serie pur interfacciandosi con il sistema parallelo.



Questi registri sono interni all'architettura. Da un lato interagiscono con la CPU usando il BUS interno e dall'altro lato con la periferica tramite il BUS esterno. Come detto in precedenza c'è anche un condizionamento di segnale. Questo adatta i segnali del mondo esterno per essere letti o scritti nella memoria DATO-STATO-CONTROLLO e permettere di adattare l'attacco esterno della periferica con la comunicazione parallela che la CPU ha con i registri.

Come facciamo ad aumentare la velocità? Fare una memoria che sia grande e veloce è quasi impossibile. Allora utilizziamo la memoria cache, in modo che la CPU parla direttamente con la cache, se all'interno non c'è qualcosa allora va nella memoria. La cache è veloce e piccolina.

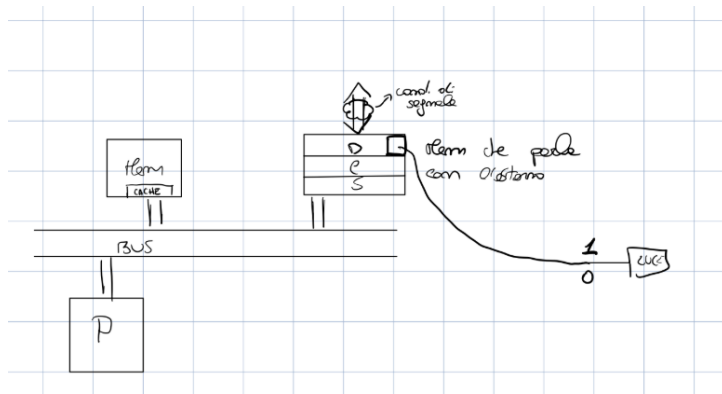
3. Metodi di Interfacciamento CPU-Periferiche

Esistono due filosofie principali per permettere alla CPU di indirizzare i registri delle periferiche:

- **Memory-mapped:** La CPU vede i registri (Dato, Stato, Controllo) come se fossero normali locazioni di memoria. Si usano i codici operativi standard (es. move in Motorola 68K) che offrono tutti i modi di indirizzamento. Il problema è che bisogna stare attenti agli indirizzi che potrebbero appartenere al set di indirizzi usati dalla memoria. Il processore non sa (e non gli interessa minimamente) se all'indirizzo \$4000 c'è un banco di memoria RAM vera o il registro di un sensore di temperatura. Lui esegue semplicemente l'accesso.
- **I/O-mapped:** il processore gestisce **due spazi di indirizzamento completamente separati**: uno per la memoria (RAM/ROM) e uno per le periferiche di I/O. Si usano codici operativi dedicati (es. In/Out).

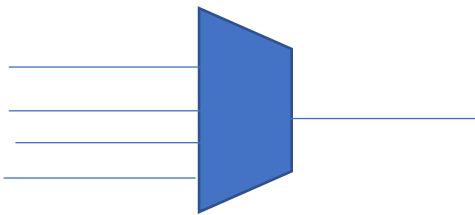
La differenza è che quelli memory-mapped, tramite la move hanno tutti i modi di indirizzamento, invece I/O-mapped no.

Se vogliamo accendere una lampadina ci conviene farlo seriale o parallelo? La lampadina ha solo bisogno di sapere accesa/spenta, quindi 1/0. Un solo bit dato, una sola informazione. Un protocollo seriale prevede l'invio di 8 bit, quindi non ci serve inviare un byte tramite comunicazione seriale. In questo caso conviene usare la comunicazione parallela.



La luce è un sistema passivo, non ha bisogno di segnali, clock ecc, vuole solo sapere accendi/spegni. Quindi il dato verrà collegato al dato del registro DSC tramite il bus esterno. Stato e controllo non saranno collegati a niente.

Sullo schedino della stm abbiamo tutte le periferiche a disposizione, questo significa quindi avere 3 fili per ognuno. Per selezionarne uno solo abbiamo bisogno di un mux



Ad esempio 4 periferiche, ogni filo porta con se 3 segnali (dato, stato, controllo). Ne possiamo selezionare uno solo.

- **Logica tristate:** è una logica che disconnette la periferica dal bus dati quando non riconosce il proprio indirizzo. In questo stato, la periferica non interferisce con le comunicazioni e ignora qualsiasi variazione presente sul bus. La logica tristate si basa sull'indirizzo interno assegnato alla periferica stessa.

Si definisce tristate perchè un'uscita può trovarsi in tre stati logici:

- 0: livello basso;
- 1: livello alto;
- Alta impedenza: praticamente disconnesso;

- **Logica Plug-and-play:** in questo caso, l'indirizzo della periferica non è fisso, ma viene selezionato dinamicamente tra un insieme di indirizzi disponibili. Questo consente di configurare automaticamente in fase di inizializzazione le periferiche senza dover intervenire manualmente per impostare l'indirizzo

4. Modalità di Comunicazione (Protocolli)

Esistono quattro modi per collegare il processore alle periferiche, basati sul coordinamento temporale:

- **Passivo:** Non c'è comunicazione di controllo; la CPU presuppone che la periferica sia sempre pronta (es. una lampadina).
- **Sincrono:** La comunicazione avviene con un **clock di riferimento** comune per entrambi.

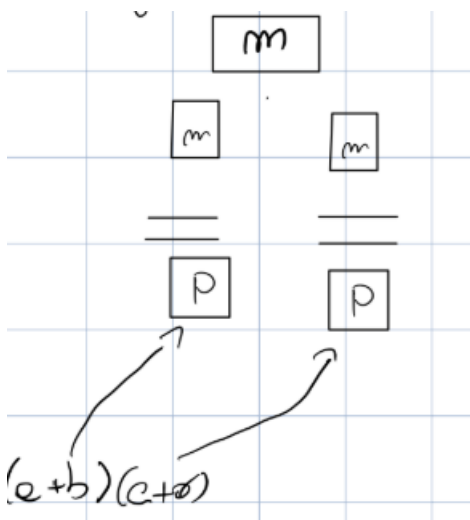
- **Handshaking (Asincrono):** Assenza di clock comune. Si usano due segnali per sincronizzare le operazioni tra CPU e periferica. Il mondo esterno non sa com'è fatto il sistema e viceversa. Sanno solo di aspettarsi dei segnali.
- **Semisincrono:** Esiste una base dei tempi comune (clock), ma si usano segnali aggiuntivi (come Start o Allineamento periodico) per mantenere il sincronismo tra trasmettitore e ricevitore.

Quando progettiamo un sistema, dobbiamo avere a disposizione CPU, memoria, Bus, registri DSC. Bisogna verificare che siano ad esempio tutti sincroni altrimenti non funzionano e inoltre valutare che tipo di modello è adottato se memory o I/O mapped

5. Evoluzione e Performance: Multi-core e Multi-computer

Per aumentare la velocità senza alzare eccessivamente la frequenza (che causerebbe danni), si usano più processori:

- **Multi-core (Multi-processor):** Più processori che lavorano insieme su una gerarchia superiore o condividono lo stesso BUS/memoria. Il coordinamento avviene tramite hardware.
- **Multi-computer:** Sistemi dove i nodi sono lontani e non possono essere collegati direttamente; devono eseguire un "binario" specifico per collaborare.



Non potremmo usare un BUS soltanto? No perché poi se uno dei due utilizza la memoria, l'altro non potrebbe, perdendo di fatto il parallelismo. Allora si usa un'altra memoria che si trova gerarchicamente ad un livello superiore, in modo tale che ogni processore ha la sua memoria e non è ostacolato dall'altro e può fare calcoli in parallelo, e poi utilizza la memoria comune se vuole comunicare con l'altro.

Se ho pochi nodi conviene usare un sistema multi-core perché sfrutto la velocità hardware. Se ho troppi core collegati al BUS posso decidere di sdoppiare e creare dei sistemi multicomputer ognuno multicore.

6. DRIVER

Le periferiche che possiamo costruire sono di 2 tipi differenti:

- Parallele: il dato che abbiamo può uscire da 1 a n bit. Il dato è sempre scritto in parallelo. Può decidere di non far uscire tutti i bit
- Seriali: il dato entra come byte ed esce come un insieme di bit.

Qualunque comunicazione seriale deve far uscire i bit uno alla volta. Un dato o esce in parallelo o in serie. Nell'ambito della serie abbiamo diverse opportunità.

Ci sono altre tre periferiche:

- DMA (direct memory access)
- PIC
- timer

Hanno un obiettivo, non gestiscono l'invio di un byte, ma l'invio di un messaggio (DMA), comunica con un'altra periferica. Si può dire che prende il posto del processore. Affidare al DMA il trasferimento di un dato efficientia il sistema, perché il processore non è occupato a trasferire il dato.

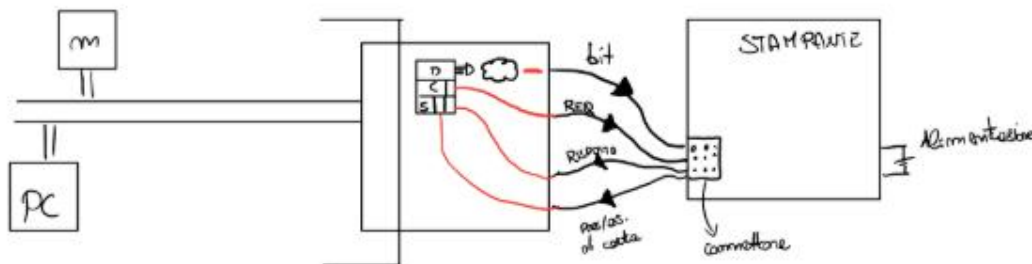
È importante perché trasferisce un messaggio: nelle reti di calcolatori, i pacchetti sulla rete. Il DMA è fondamentale nelle reti.

PIC è un oggetto che serve a gestire le interruzioni. Le interruzioni sono di vario tipo. PIC non gestisce tutti i tipi di interruzioni, solo quelle che vengono dall'esterno ovvero dalle periferiche. DMA e PIC non hanno una vita propria, relative all'I/O. servono in quanto esiste un oggetto esterno. Possono lavorare anche insieme.

Un'architettura potrebbe avere parallelo, DMA, PIC. Normalmente ci sono tutte e 3.

Utilizzare queste 3 periferiche insieme, il simulatore non funziona molto bene.

Supponiamo che vogliamo collegare una stampante (che si collega in serie) con un computer per trasmettere un carattere. Questo lo vogliamo fare tramite un'interfaccia.



Architettura Hardware e Interfaccia Periferiche

L'interfaccia tra il processore e la stampante si basa su un modello DATO-STATO-CONTROLLO. *Il byte non può viaggiare direttamente sul filo dato perché l'uscita è di tipo seriale, mentre il bus parla sempre parallelo*, per questo motivo, occorre inserire un oggetto (come una UART) in grado di prendere il dato in parallelo e farlo uscire in seriale.

Il bit di controllo è collegato fisicamente alla richiesta (REQ).

Il segnale che indica la presenza o l'assenza di carta viene posizionato sullo STATO, sul bit 1 del registro stato di 8 bit. Quindi se c'è assenza di carta il registro sarà del tipo 0x02 in binario 00000010

Le periferiche più complesse sono dotate di un ulteriore registro: registro MODO per dare la possibilità di configurare i parametri di trasferimento in serie (ad esempio trasferimento a 5 bit, 6 bit, ecc.).

Se ad esempio avessi una lavatrice invece della stampante, il dato sarebbe il programma da selezionare che non è da confondere con il MODO, ma indica solo la modalità di funzionamento, non il modo di trasferimento del dato. Il MODO è una scelta esclusiva del processore: cambiandolo, cambia il

funzionamento della periferica. La funzione del MODO è unicamente quella di settare la configurazione del trasferimento seriale.

Tramite il modello memory-mapped, le periferiche vengono mappate direttamente nello spazio di memoria, usando la direttiva ORG \$20000, ci si aspetta di trovare le periferiche a partire da quell'indirizzo.

Occorre però assicurarsi che a tali indirizzi risiedano effettivamente le periferiche e non la memoria (RAM). L'istruzione ORG \$20000 rappresenta una "promessa": funziona solo se l'hardware è rispettoso di tale promessa e che cioè non abbia destinato tale indirizzo alla memoria.

Allocazione ipotizzata: DATO all'indirizzo 20000, CONTROLLO a 20001, STATO a 20002.

In realtà, STATO e CONTROLLO possono condividere lo stesso indirizzo (es. 20001): se il processore vi scrive, sta usando il controllo; se lo legge, sta ottenendo lo stato, come la PIA.

Le definizioni di spazio per i registri sono DATO DS.B 1, CONTR DS.B 1 e STATO DS.B 1.

È fondamentale avere la possibilità di leggere lo stato attuale senza prendere per sbaglio quello precedente.

Immaginiamo che all'inizio non ci sia carta quindi il registro stato è 00000010. Noi se non c'è carta ci aspettiamo che sia proprio questo quindi facciamo la AND tra questo e il valore dello stato. La AND fa AND bit a bit, cioè è una maschera il valore che vogliamo noi. Non ci interessa degli altri bit. Se il registro stato fosse 10010010 succede che:

00000010 // valore che vogliamo vedere

AND

10010010 //registro stato copiato in D0

Risultato: 00000010 e viene salvato in D0, ovvero sempre nel secondo operando.

Questo numero è diverso da 0 significa che il bit di nostro interesse è 1.

BNE salta se il risultato dell'istruzione precedente è diverso da 0. Infatti se è diverso da 0 non c'è carta e va a FINE.

La stessa cosa per vedere se ha alzato il bit di stato. Questa volta dobbiamo controllare il bit 0. Se la and da 0 significa che dobbiamo ricontrollare. Per questo usiamo BEQ cioè se uguale a 0 fai qualcosa.

```
ORG $2000
```

```
DATO DS.B 1
```

```
CONTROLLO DS.B 1
```

```
STATO DS.B 1
```

```
ORG $1000
```

```
VAR DC.B 48 //Valore che vogliamo inviare alla stampante.
```

```
ORG $8000
```

```
MOVE.B #0, STATO
```

```
MOVE.B #0, CONTROLLO
```

```
MOVE.B STATO, D0
```

Fase di inizializzazione: è importante partire da uno stato conosciuto, quindi metto tutto a 0. Poi mi porto lo stato in D0 per il controllo sulla carta, perché se da subito c'è assenza di carta, è inutile chiedere di poter stampare qualcosa. Mi fermo subito.

```
AND.B #$02, D0 //se è 0 posso inviare, perché isola il bit 1
```

```
BNE FINE // se non c'è carta, salta alla fine
```

```
MOVE.B #$01, CONTROLLO //assicurati che c'è carta, richiediamo di inviare
```

```
L1 MOVE.B STATO, D0
```

Ciclo di attesa dovuto al protocollo di handshaking

```
AND.B #$01, D0 // isola il bit 0 dello stato
```

```
BEQ L1
```

```
MOVE.B VAR, DATO //il bit stato è diventato 1, invio effettivo del dato
```

```
MOVE.B #0, CONTROLLO //resettiamo
```

```
MOVE.B #0, STATO
```

Protocollo di Comunicazione (Handshaking)

Il sistema sfrutta un protocollo per trasmettere in seriale, che utilizza l'handshaking sui segnali di controllo/risposta (RISP). Il registro di STATO a 8 bit impiega due fili che portano 1 singolo

bit ciascuno: il bit 1 indica la presenza/assenza di carta, il bit 0 indica il filo stato che dalla stampante va verso il registro. Quindi due fili sono collegati allo stesso registro ma su due bit differenti.

La notazione con il simbolo \$ indica un valore esadecimale (es. \$02 corrisponde al binario 00000010).

Applicando una maschera logica, se nella posizione 1 c'è il valore 1, significa che non c'è carta e il protocollo termina (FINE). Se altrimenti c'è 0 il sistema può inviare il dato, poiché c'è presenza di carta.

Il processore, quindi, avvisa la stampante che è pronta settando ad 1 il bit Controllo. A questo punto si entra in un'attesa potenzialmente infinita, aspettando che la stampante metta il proprio Stato a 1 per confermare la ricezione; solo allora si invia il dato.

Dopo aver inviato il dato, si resetta tutto.

Ipotesi software (successivamente rimossa): appena il sistema mette il dato nel registro, questo lo manda subito nel modo che conosce.

Ottimizzazione del Driver e Indirizzamento

Il driver scritto così presenta alcune criticità: un driver scritto con indirizzi assoluti non è riutilizzabile: se si compra un'altra stampante, non lo si può riutilizzare, perché gli indirizzi rimangono quelli. Non sappiamo se sarà a 2000, 3000 etc. abbiamo necessità di generalizzare. Quando facciamo move.b stato, D0, stato è all'indirizzo 2000. Ma se un'altra stampante viene collegata e messa all'indirizzo 4000, DATO si troverà all'indirizzo 4000 e non si può più riutilizzare. Per ovviare a questo problema, è necessario utilizzare l'indirizzamento relativo.

Passando dinamicamente l'indirizzo dei 3 registri al driver, questo diventa universale e riutilizzabile. In questo modo se la stampante 1 ha il dato a 2000 passo l'indirizzo di DATO, se sta a 4000 passo 4000.

Le istruzioni Assembly usate per l'indirizzamento relativo sono:

```
MOVEA.L #DATO, A0  
  
MOVEA.L #STATO, A1  
  
MOVEA.L #CONTROLLO, A2
```

```
* --- DEFINIZIONE INDIRIZZI STAMPANTE 1 ---
```

```
ORG $20000
```

```
DAT01      DS.B 1
```

```
CONTROLLO1 DS.B 1
```

```
STAT01     DS.B 1
```



```

* --- DEFINIZIONE INDIRIZZI STAMPANTE 2 ---

ORG $40000          ; La seconda stampante fisicamente è qui!

DAT02      DS.B 1

CONTROLLO2 DS.B 1

STAT02     DS.B 1


* =====

* --- INIZIO PROGRAMMA PRINCIPALE ---

* =====

ORG $8000

* >> VOGLIO USARE LA STAMPANTE 1 <<

* Fase di setup per la prima stampante:

MOVEA.L #DAT01, A0      ; A0 ora punta a $20000

MOVEA.L #STAT01, A1     ; A1 ora punta a $20002

MOVEA.L #CONTROLLO1, A2 ; A2 ora punta a $20001

JSR DRIVER_STAMPANTE    ; Salta al Driver ed esegue la stampa!


* >> VOGLIO USARE LA STAMPANTE 2 <<

* Rifaccio il setup, ma per la seconda stampante:

MOVEA.L #DAT02, A0      ; A0 ora punta a $40000

MOVEA.L #STAT02, A1     ; A1 ora punta a $40002

MOVEA.L #CONTROLLO2, A2 ; A2 ora punta a $40001

JSR DRIVER_STAMPANTE    ; Richiamo lo STESSO IDENTICO Driver!


FINE_PROGRAMMA: ...      ; Fine del programma principale


* =====

* --- INIZIO DEL DRIVER (SUBROUTINE) ---

```

```

* =====

DRIVER_STAMPANTE:

MOVE.B (A1), D0          ; Leggo lo Stato dal registro in A1

AND.B #$02, D0           ; Controllo se c'è carta

BNE FINE                 ; Se non c'è, fine.


* CICLO DI POLLING

MOVE.B #$01, (A2)        ; Avviso che sono pronto sul Controllo (A2)

L1: MOVE.B (A1), D0       ; Leggo di nuovo lo Stato (A1)

AND.B #$01, D0

BEQ L1                   ; Aspetto finché non è pronta...


MOVE.B VAR, (A0)          ; Mando il dato nel registro puntato da A0!


FINE:

RTS                      ; Return To Subroutine (Torna al programma principale)

```

Il driver lo scrivo una volta solo e posso usarlo quante volte voglio con le diverse configurazioni delle diverse stampanti.

Evoluzione del Sistema: Interruzioni, DMA e Timer

Ci sono altre criticità nel driver così progettato: *l'approccio basato sull'attesa continua (polling) rischia di tenere il sistema bloccato all'infinito*. Per risolvere questa problematica facciamo uso di interruzioni: Inserire le interruzioni risolve il problema delle attese legate al protocollo. Se abbiamo più periferiche collegate, come facciamo a sapere chi ha inviato una certa interruzione e chi un'altra? La soluzione è l'uso di una periferica che si chiama PIC.

PIC: Si usa sul registro di stato per gestire il meccanismo delle interruzioni. Anche se non risolve direttamente il problema, offre il vantaggio di dire chi ha interrotto il processore in un sistema con più periferiche; se si sa chi è, si sa cosa fare. Senza il PIC il processore dovrebbe andare nel registro stato e controllare tutti i bit per capire chi lo ha interrotto.

DMA: Introducendo il DMA, possiamo inviare in unico messaggio tutti i dati. Se si vogliono inviare 100 caratteri, è sufficiente mandare il messaggio al DMA e questo gestisce autonomamente "100 Req". Senza il DMA dovrei eseguire 100 volte il programma.

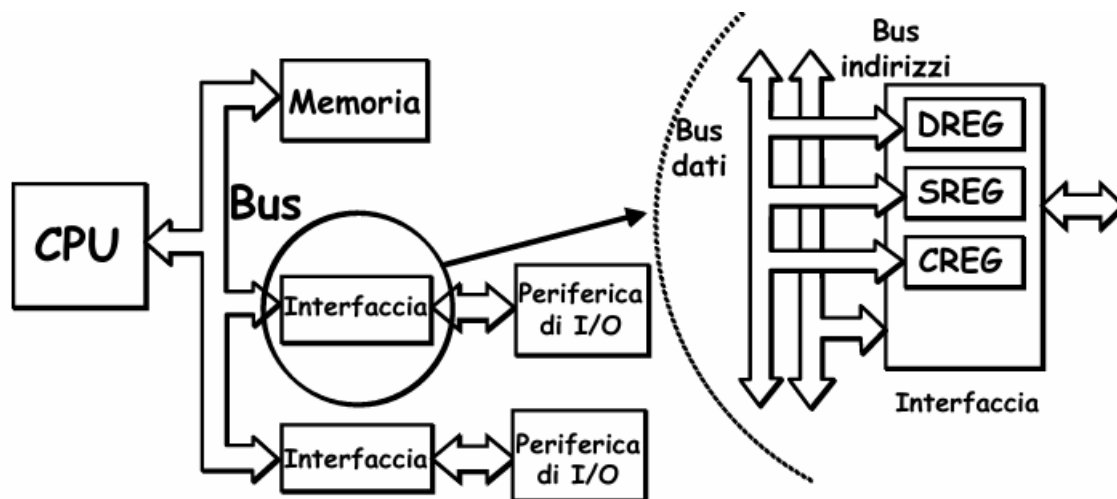
Se la periferica ha un qualunque tipo di mal funzionamento e quindi non aggiorna mai il registro di stato, tale ciclo eseguirà all'infinito senza mai fermarsi.

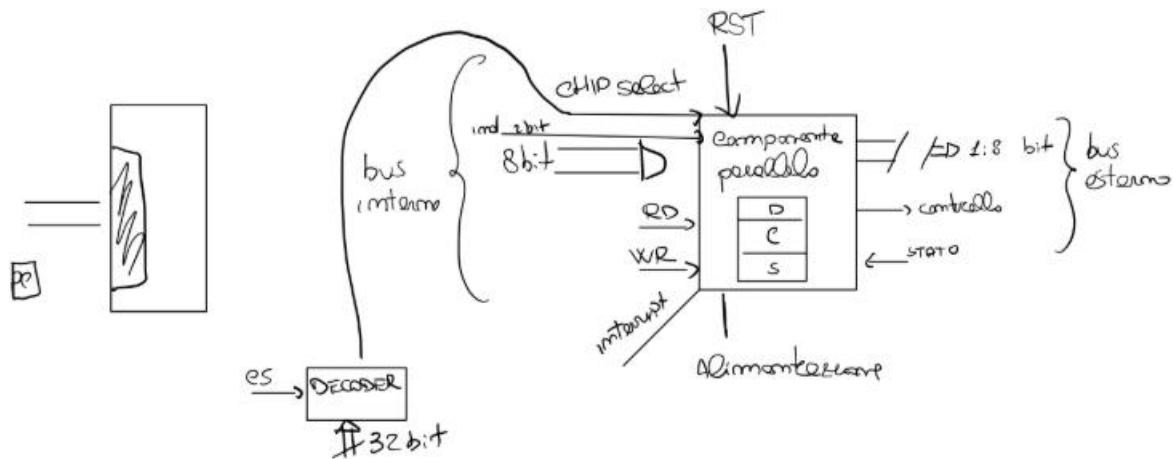
Dato che le periferiche operano in modo asincrono a velocità di solito assai minore di quella della CPU, esiste il problema di come sincronizzare lo scambio dei dati tra il processore e le unità esterne. Ci sono tre approcci fondamentali:

1. polling o controllo da programma;
2. uso di un sistema di interruzioni;
3. uso di un processore di I/O (DMA).

Timer: Il sistema necessita anche di un timer, e ne serve più di uno nel caso siano collegate diverse periferiche.

Tuttavia, se si possiede un timer dedicato, il DMA potrebbe non servire.





Il componente parallelo è l'adattatore (interfaccia) che si posiziona tra la periferica e il processore. Il bus interno è collegato sullo stesso BUS con il quale il processore comunica con la memoria. Il bus esterno è quello che si collega con il connettore della periferica. L'interfaccia di I/O è costituita dalla logica necessaria a trasferire i dati fra bus a dispositivo. Dal lato del dispositivo di I/O può essere di tipo PARALLELA o SERIALE, dal lato del bus è sempre parallela. È semplicemente una vista più dettagliata della seconda memoria che abbiamo aggiunto all'architettura del processore.

Poiché il componente è attaccato allo stesso BUS della Memoria, il Processore comunica con lui **esattamente come se fosse un pezzo di RAM**. Quando il processore vuole mandare un dato alla periferica, non fa altro che scrivere un valore in un indirizzo di memoria specifico (es. 200), che il decoder riconoscerà, "svegliando" questo blocco D, C, S invece della memoria principale. Questa tecnica si chiama *Memory-Mapped I/O*. Ogni periferica ha il suo decoder.

L'interfaccia di I/O è il circuito che collega la periferica al Bus di sistema e vengono chiamate porte. Spesso le porte si trovano nel calcolatore e le periferiche si collegano ad esse tramite connettori di vario tipo (lato periferica della porta).

Bus Interno: Si deve collegare al processore. Comprende 2 bit per gli indirizzi, 8 bit per i dati e segnali di controllo come RD (Read) e WR (Write).

Bus Esterno: il dato va da 1 a 8 bit, poi c'è controllo e stato per fare l'handshaking con la periferica. Se la comunicazione fosse seriale avrei bisogno invece di questa interfaccia

Segnali di Controllo: Oltre a RD e WR, i segnali includono RST (Reset), CHIP Select, interrupt e l'alimentazione. Processore e periferica si devono scambiare segnali per indicare che il bus è collegato.

Gestione degli Indirizzi: Gli indirizzi di memoria del sistema sono a 32 bit. Tuttavia, il decoder riceve 30 bit per determinare la posizione e usa i restanti 2 bit per selezionare uno specifico registro (00, 01, 10). I 2 bit servono per indicare quale dei tre registri selezionare (ad esempio, assegnando DATO a 200, CONTROLLO a 201 Il Decoder funge da "portiere di condominio":

La maggior parte dei bit (es. i 30 bit più significativi) entra nel Decoder. Il Decoder è un circuito logico programmato (o cablato) per riconoscere uno specifico indirizzo di base. Se l'indirizzo messo dalla CPU sul bus corrisponde alla "zona" assegnata alla PIA (es. la zona \$200X), il Decoder si attiva e accende un singolo filo chiamato CHIP SELECT (CS).

Questo filo CS entra nella PIA e la "sveglia.

I numero di bit che "scavalcano" il decoder e vanno diretti dentro il componente dipende dalla complessità del componente stesso. Questi bit si chiamano Register Select (RS_0, RS_1 etc).

La parte alta (i bit più significativi) va al Decoder per generare il Chip Select e svegliare il chip giusto. **La parte bassa** (gli ultimi bit) va direttamente dentro il chip svegliato per scegliere quale registro interno leggere o scrivere. Quanti siano questi bit nella parte bassa dipende solo da quanti registri ha il chip!

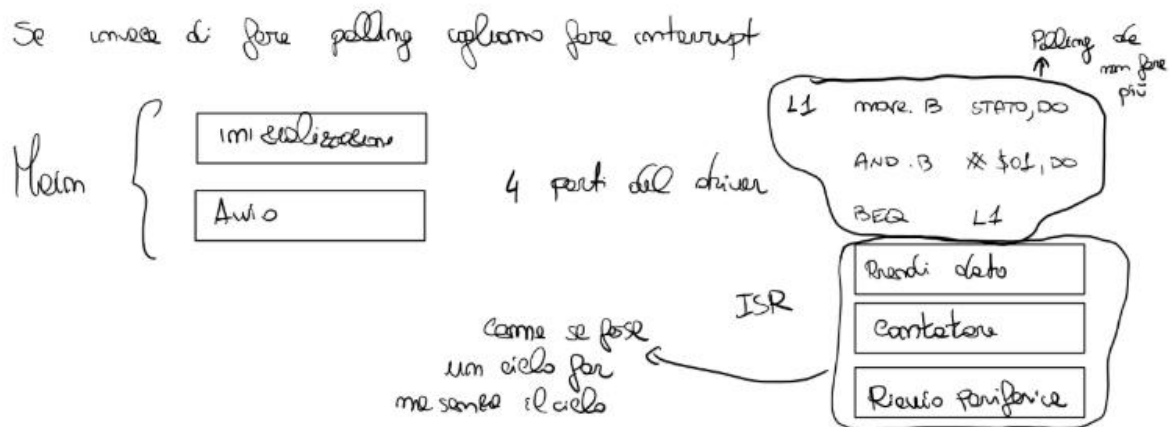
Nel codice del driver per vedere se la periferica rispondeva con stato = 1 facevamo un ciclo potenzialmente infinito secondo un meccanismo chiamato polling.

```
L1    MOVE.B STATO, D0  
  
      AND.B #$01, D0 // isola il bit 0 dello stato  
  
      BEQ L1
```

Il codice di polling (che verifica il registro di STATO) può durare all'infinito e impedisce al processore di eseguire altre operazioni. Per evitare attese infinite durante il polling, si può inserire un Timer. Se non si vuole usare il polling per non bloccare il processore, si usa l'interrupt. Quando la periferica è pronta, invia l'interrupt.

*La disponibilità di un sistema di interruzioni permette alla CPU di svolgere altre operazioni mentre le unità periferiche sono attive. Quando una periferica necessita della CPU, ad esempio perché ha appena prodotto un dato o ricevuto un segnale dall'esterno, invia alla CPU una richiesta di intervento. Questo segnale viene indicato con il termine *interruzione, perché, di norma, la sua ricezione interrompe l'esecuzione del programma da parte della CPU*, che comincia ad occuparsi della periferica che lo ha inviato.*

Sfrutta un registro di stato delle interruzioni (ISR), ciascun bit del quale è associato ad una periferica. Una periferica che richieda una interruzione asserisce al tempo stesso il proprio bit del registro. Al recepimento della richiesta, la routine di servizio esamina sequenzialmente i bit di ISR e determina quale periferica ha attivato la richiesta. Di conseguenza, attiva (e.g. con una istruzione CALL) il sottoprogramma di gestione opportuno. Questa soluzione realizza implicitamente anche un meccanismo di priorità, in quanto i bit di ISR vengono letti in un ordine fissato e nello stesso ordine vengono servite eventuali richieste di interruzione concorrenti. La gestione di ISR è a carico del programma, che deve abbassare i bit delle interruzioni servite e le stesse richieste, in modo da evitare loop pericolosi.



Fasi Obbligatorie: Le fasi di "Inizializzazione" e "Avvio" devono essere eseguite per forza. L'unica parte che cambia è il codice relativo al polling, che non viene più utilizzato. Se si devono acquisire più caratteri, è necessario un contatore, che conta i caratteri e poi riavviare la periferica. Questo meccanismo è simile ad un ciclo for, con la differenza che non è un ciclo esplicito, ma implicito, perché quando il contatore è arrivato al numero di caratteri che si aspetta di ricevere farà BEQ #5, Cont FINE.

Se però si inserisce il DMA, il contatore non serve più, perché gestisce stesso lui l'invio di messaggi.

Se ci sono più periferiche che possono generare interrupt, per capire chi ha interrotto il processore si fa affidamento ai bit di stato. In questo caso si esegue un "polling proattivo", diverso dal vero polling, che consiste nello scorrere il registro di stato per trovare il primo bit impostato a 1.

7. Interruzioni, Eccezioni e Stato del Processore

Classificazione: Le interruzioni si dividono in interne ed esterne. Le interruzioni interne prendono il nome di *eccezioni*. Quelle esterne (chiamate semplicemente *interruzioni*) si verificano quando una periferica esterna interrompe il sistema.

TRAP e Modalità Supervisore: La TRAP è un codice operativo che *permette il passaggio dallo stato utente allo stato supervisore*. Questo serve per accedere a registri protetti o gestire interruzioni interne, come ad esempio scrivere in memoria.

Sistemi Reali: In un sistema reale, per compiere queste operazioni, si dovrebbe necessariamente passare prima in stato supervisor.